===== 1. OVERVIEW =====
CloudSystem is a Spring Boot 4.1.1 microservice for distributed cloud file storage.
Written in Java 17, using Spring Data JPA + PostgreSQL for persistence,
JWT (Auth0 library) for stateless authentication, and STOMP WebSocket for node communication.

WHAT IT DOES TODAY:
- User registration/login with BCrypt password hashing + JWT token
- File upload: stores metadata in DB, computes SHA-256 checksum
- Storage node registration + heartbeat tracking
- File replication: creates replica records for 3 online nodes (PENDING status)

WHAT IT DOES NOT DO YET (MVP placeholders):
- Actual file bytes transfer to storage nodes (replicas are metadata-only)
- ContainerService.buildContainer/extractContainer are passthrough stubs
- File download returns empty byte array
- No WebSocket message handlers implemented
- UserServiceImpl.getCurrentUser() returns null
- JWT secret is hardcoded in source code
===== 2. ARCHITECTURE & FLOW =====
Request lifecycle for every REST call:
> HTTP Request -> SecurityFilter (JWT validation) -> Controller -> Service -> Repository -> DB

SecurityFilter (OncePerRequestFilter):
1. Reads Authorization header
2. If Bearer token present, validates via TokenConfig.validateToken()
3. If valid: creates UsernamePasswordAuthenticationToken with JWTUserData,
   sets it in SecurityContextHolder.getContext().setAuthentication()
4. If invalid/missing: does NOT set authentication -> request proceeds unauthenticated
   (but SecurityConfig requires auth for all non-/auth endpoints -> 403)

CRITICAL: filterChain.doFilter() is called in BOTH branches (valid token and no token),
so the filter always lets the request proceed. The difference is whether the SecurityContext
has an Authentication object. @AuthenticationPrincipal on controller methods extracts it.
===== 3. SECURITY ARCHITECTURE (DETAILED) =====
>> 3.1 SecurityConfig
@Configuration @EnableWebSecurity
- CSRF disabled (stateless REST API)
- CORS configured for localhost:5173, 127.0.0.1:5173, 127.0.0.1:5500
- Session policy: STATELESS (no HttpSession)
- PermitAll: POST /auth/login, POST /auth/register, ERROR dispatcher
- Authenticated: ALL other requests
- Filter chain: SecurityFilter added BEFORE UsernamePasswordAuthenticationFilter
- Beans: AuthenticationManager, BCryptPasswordEncoder, CorsConfigurationSource
>> 3.2 SecurityFilter
@Component, extends OncePerRequestFilter (runs once per request)
Reads 'Authorization' header -> extracts Bearer token -> validates -> sets context.
BUG: If token is present but INVALID (expired/wrong), it silently passes through
because filterChain.doFilter() is called in the else branch regardless.
The authentication just won't be set, and the endpoint will return 403.
>> 3.3 TokenConfig
@Component - JWT encode/decode using HMAC256 (secret hardcoded)
generateToken(User): claims={userID}, subject=email, expires=24h, issuedAt=now
validateToken(String): returns Optional<JWTUserData> or empty on JWTVerificationException
WARNING: secret 'Evgueny25087Nenane' is in source code - should be env var
>> 3.4 AuthConfig
@Service implements UserDetailsService
loadUserByUsername(email) -> userRepository.findUserByEmail(email)
Used by Spring Security's AuthenticationManager during login
>> 3.5 JWTUserData
@Builder record(userID: Long, email: String)

Carries user identity after JWT validation. Used as 'principal' in SecurityContext.
===== 4. ENTITY CLASSES (DETAILED) =====
>> 4.1 User implements UserDetails
- @Entity(name='Users') -> table 'Users'
- userID: Long PK AUTO_INCREMENT
- name, surname, email: String(100), NOT NULL
- password: String, NOT NULL (BCrypt encoded)
- registrationData: LocalDateTime, NOT NULL
- lastAccess: LocalDateTime (nullable, not currently set anywhere)
- role: @ManyToOne Role, FK roleID
UserDetails methods:
- getAuthorities() -> ['ROLE_' + role.getName()] (or empty if role==null)
- getUsername() -> email
- getPassword() -> inherited from entity -> password field
- isAccountNonExpired/NonLocked/CredentialsNonExpired -> default true (from UserDetails)
NOTE: User does NOT implement getPassword() explicitly - Spring Security uses
the password field directly via reflection since it's a JPA entity.
>> 4.2 Role
- @Entity(name='Roles') -> table 'Roles'
- roleID: Long PK, name: String unique NOT NULL
- users: @OneToMany mappedBy='role' (inverse side)
>> 4.3 StorageNode
- @Entity(name='StorageNodes') -> table 'StorageNodes'
- nodeID PK, owner @ManyToOne User (FK ownerID)
- name String(100) NOT NULL
- status NodeStatus enum DEFAULT OFFLINE
- totalSpaceBytes NOT NULL, usedSpaceBytes default 0
- lastHeartbeat, sessionId, createdAt
- getFreeSpaceBytes() = totalSpaceBytes - usedSpaceBytes
NOTE: 'createdAt' is never set in NodeController.registerNode() - will be NULL in DB!
>> 4.4 FileMetadata
- @Entity(name='FileMetadata') -> table 'FileMetadata'
- fileID PK, owner @ManyToOne User
- originalName(255), mimeType(150), originalSizeBytes, storedSizeBytes
- checksumSha256(64) - SHA-256 hex of raw file bytes
- containerVersion default 1
- encryptedDataKey TEXT (AES-256 key, Base64 - NOT actually encrypted in MVP)
- keyEncryptionIv String(64)
- uploadedAt NOT NULL
- replicas @OneToMany mappedBy='file', cascade=ALL, orphanRemoval=true
NOTE: storedSizeBytes = originalSizeBytes in MVP (no actual compression/encryption)
NOTE: encryptedDataKey/keyEncryptionIv are stored but not actually used for encryption
>> 4.5 FileReplica
- @Entity(name='FileReplicas') -> table 'FileReplicas'
- replicaID PK, file @ManyToOne FileMetadata, node @ManyToOne StorageNode
- status ReplicaStatus DEFAULT PENDING, remotePath, updatedAt
This is the link table between FileMetadata and StorageNode with status tracking.
>> 4.6 Enums
- NodeStatus: ONLINE, OFFLINE, DEGRADED
- ReplicaStatus: PENDING, SYNCED, FAILED
===== 5. REPOSITORY LAYER =====
All extend JpaRepository<T, Long> -> standard save/findAll/findById/delete
>> UserRepository
- findUserByEmail(String) -> Optional<UserDetails>
Used by AuthConfig (UserDetailsService) and AuthController (email uniqueness check)
>> RoleRepository
- Standard CRUD only
Used by AuthController to assign role ID 3 on registration
>> StorageNodeRepository
- findAllByOwner(User) - all nodes owned by a user

- findBySessionId(String) - find node by session
- findAllByStatusAndLastHeartbeatBefore(NodeStatus, LocalDateTime) - stale nodes
- findAllByStatus(NodeStatus) - all nodes with given status
- findAllByStatusOrderByUsedSpaceBytesAsc(NodeStatus) - nodes sorted by used space
>> FileMetadataRepository
- findAllByOwner(User) - all files for a user
- findByFileIDAndOwner(Long, User) - specific file + owner check (security)
>> FileReplicaRepository
- findAllByFile(FileMetadata)
- findAllByFileAndStatus(FileMetadata, ReplicaStatus)
- findAllByNode(StorageNode)
- findByFile_FileIDAndNode_NodeID(Long, Long) - specific replica lookup
===== 6. SERVICE LAYER (DETAILED) =====
>> 6.1 ContainerService
@Service - handles file metadata and container logic.
- uploadFile(User owner, MultipartFile file) -> Long:
1. Reads file.bytes
2. Computes SHA-256 hex of raw bytes
3. Builds FileMetadata with originalName, mimeType, sizes, checksum, uploadedAt
4. Saves to DB via fileMetadataRepository.save()
5. Returns generated fileID
- buildContainer(byte[], String, String, Long) -> byte[]:
MVP STUB - returns raw data unchanged. Future: wrap in container format
- extractContainer(byte[], User) -> byte[]:
MVP STUB - returns data unchanged. Future: unwrap container
- findById(Long) -> FileMetadata:
Throws RuntimeException if not found
>> 6.2 ReplicationService
@Service @RequiredArgsConstructor - async replication.
- replicateFile(Long fileID) [(@Async, @Transactional)]:
1. Loads FileMetadata by fileID
2. Finds all ONLINE StorageNodes
3. Selects top 3 by freeSpaceBytes descending
4. If <3 nodes available: logs warning, returns (NO REPLICATION)
5. For each selected node: creates FileReplica(entity) with PENDING status
6. Saves each replica to DB
IMPORTANT: No actual data transfer happens - this only creates DB records
- handleNodeAck(Long fileID, Long nodeID, ReplicaStatus status):
Finds replica by fileID+nodeID, updates status and updatedAt
Used when a storage node confirms replication completion
- selectNodesWithSpace(List<StorageNode>, int) -> List<StorageNode>:
Private helper: sorts by freeSpaceBytes desc, limits to factor count
>> 6.3 UserService / UserServiceImpl
UserService interface: getCurrentUser() -> User
UserServiceImpl: returns null - placeholder for SecurityContextHolder integration
Used to get the authenticated user in service layer (not yet wired)
===== 7. CONTROLLER LAYER (DETAILED) =====
>> 7.1 AuthController @RequestMapping('/auth')
Constructor-injected: UserRepository, RoleRepository, PasswordEncoder,
AuthenticationManager, TokenConfig
- POST /auth/login (@RequestBody LoginRequest):
1. Creates UsernamePasswordAuthenticationToken(email, password)
2. authenticationManager.authenticate() -> loads User via AuthConfig
3. Compares BCrypt password -> throws exception if mismatch
4. Gets User from authentication.getPrincipal()
5. Generates JWT via tokenConfig.generateToken(user)
6. Returns 200 OK + LoginResponse(token)
- POST /auth/register (@RequestBody RegisterUserRequest):
1. Creates new User entity, encodes password with BCrypt
2. Sets registrationData = LocalDateTime.now()

3. Assigns role = roleRepository.getReferenceById(3L) // Role ID 3 = default
4. Checks email uniqueness via isEmailUnique() -> returns 409 if duplicate
5. Saves user, generates JWT
6. Returns 201 CREATED + RegisterUserResponse(name+surname, email, token)
BUG: isEmailUnique() returns true when email EXISTS (confusing naming)
Called BEFORE save() - check happens, but if duplicate, returns 409
>> 7.2 UserController
GET /test -> returns 'Aqui esta o user controller' (test endpoint)
>> 7.3 FileController @RequestMapping('/files')
Constructor-injected: ContainerService, FileMetadataRepository, ReplicationService
- POST /files/upload (@AuthenticationPrincipal User user, @RequestParam MultipartFile file):
1. Calls containerService.uploadFile(user, file) -> fileID
2. Calls replicationService.replicateFile(fileID) [async]
3. Returns UploadResponse(fileID, filename, contentType, size)
- GET /files (@AuthenticationPrincipal User user):
1. fileMetadataRepository.findAllByOwner(user)
2. Maps each to FileMetadataResponse (including replica info)
3. Returns List<FileMetadataResponse>
- GET /files/{fileId}/download:
MVP STUB - returns empty byte array with Content-Disposition header
Real implementation would read bytes from the storage node
>> 7.4 NodeController @RequestMapping('/nodes')
Constructor-injected: StorageNodeRepository
- POST /nodes/register (@RequestParam name, @RequestParam totalSpaceBytes, @AuthenticationPrincipal User):
1. Checks if user already has a node via findAllByOwner(currentUser)
2. If exists: returns existing node (200 OK)
3. If not: builds StorageNode with status=OFFLINE, lastHeartbeat=now
NOTE: createdAt is NEVER set -> will be NULL in DB!
4. Saves and returns node
- GET /nodes:
Returns all StorageNodes for current user
- POST /nodes/{nodeId}/heartbeat (@PathVariable Long nodeId):
1. Finds node by ID
2. Sets status=ONLINE, lastHeartbeat=now
3. Saves node
NOTE: Does NOT verify the node belongs to the authenticated user!
SECURITY RISK: Any authenticated user can heartbeat any node by ID
===== 8. DTO LAYER =====
>> Request DTOs (records)
- LoginRequest(email @NotEmpty, password @NotEmpty)
- RegisterUserRequest(name @NotEmpty, surname @NotEmpty, email @NotEmpty, password @NotEmpty)
>> Response DTOs (records)
- LoginResponse(token)
- RegisterUserResponse(name, email, token)
- UploadResponse(fileID, originalName, contentType, size)
- FileMetadataResponse(fileID, originalName, mimeType, originalSizeBytes, uploadedAt, List<ReplicaInfo>)
- ReplicaInfo(nodeID, nodeName, status)
FileMetadataResponse.fromEntity(): static factory that maps FileMetadata + its replicas
to the DTO, handling null node references gracefully ('Unknown' name)
===== 9. CONFIGURATION LAYER =====
>> CorsConfig
@Configuration, defines WebMvcConfigurer bean for /auth/** CORS from 127.0.0.1:5173
>> WebSocketConfig
@Configuration @EnableWebSocketMessageBroker
STOMP endpoint /ws-nodes, simple broker /topic + /queue, app prefix /app
NOTE: No message handlers (@MessageMapping) implemented yet
>> JWTUserData
@Builder record(userID, email) - carries validated user info through security context
===== 10. COMPLETE WORKFLOWS =====
>> 10.1 User Registration (full step-by-step)

```
    > POST /auth/register {name, surname, email, password}
  1. RegisterUserRequest validated (@NotEmpty)
  2. New User entity created
  3. password = BCryptPasswordEncoder.encode(password)
  4. registrationData = LocalDateTime.now()
  5. role = roleRepository.getReferenceById(3L)  // assumes role 3 exists in DB
  6. isEmailUnique(email) checked -> 409 if exists
  7. userRepository.save(newUser)  // generates userID
  8. tokenConfig.generateToken(newUser)  // JWT with userID claim
  9. Returns 201 + RegisterUserResponse
ASSUMPTION: Role with ID=3 must exist in the database beforehand
  >> 10.2 File Upload (full step-by-step)
    > POST /files/upload + MultipartFile + Bearer JWT
  1. SecurityFilter validates JWT -> sets SecurityContext
  2. @AuthenticationPrincipal User extracted from SecurityContext
  3. ContainerService.uploadFile():
     a. file.getBytes() -> raw bytes
     b. sha256Hex(rawData) -> checksum
     c. FileMetadata.builder() with all fields
     d. fileMetadataRepository.save(metadata)
     e. returns saved.getFileID()
  4. ReplicationService.replicateFile(fileID) [async, @Transactional]:
     a. Loads FileMetadata
     b. nodeRepository.findAllByStatus(ONLINE)
     c. Sorts by freeSpaceBytes desc, takes top 3
     d. If <3 nodes: logs warning, aborts
     e. For each node: FileReplica.builder().file(metadata).node(node).status(PENDING).save()
  5. Returns UploadResponse(fileID, filename, contentType, size)
  >> 10.3 Node Heartbeat (full step-by-step)
    > POST /nodes/{nodeId}/heartbeat + Bearer JWT
  1. nodeRepository.findById(nodeId)
  2. If found: set status=ONLINE, lastHeartbeat=now, save
  3. If not found: return 404
SECURITY NOTE: No ownership check - any authenticated user can heartbeat any node
 ===== 11. DATABASE SCHEMA (PostgreSQL) =====
spring.datasource.url=jdbc:postgresql://localhost:5432/storagearchivedb
spring.jpa.hibernate.ddl-auto=update  // auto-creates/updates tables

Table: Users
  userID BIGSERIAL PK, name VARCHAR(100), surname VARCHAR(100),
  email VARCHAR(100) NOT NULL, password VARCHAR NOT NULL,
  registrationData TIMESTAMP NOT NULL, lastAccess TIMESTAMP,
  roleID BIGINT FK -> Roles.roleID

Table: Roles
  roleID BIGSERIAL PK, name VARCHAR NOT NULL UNIQUE

Table: StorageNodes
  nodeID BIGSERIAL PK, ownerID BIGINT FK -> Users.userID,
  name VARCHAR(100) NOT NULL, status VARCHAR(20) DEFAULT 'OFFLINE',
  totalSpaceBytes BIGINT NOT NULL, usedSpaceBytes BIGINT DEFAULT 0,
  lastHeartbeat TIMESTAMP, sessionId VARCHAR(100),
  createdAt TIMESTAMP  // NOTE: NULL in current code!

Table: FileMetadata
  fileID BIGSERIAL PK, ownerID BIGINT FK -> Users.userID,
  originalName VARCHAR(255) NOT NULL, mimeType VARCHAR(150),
  originalSizeBytes BIGINT NOT NULL, storedSizeBytes BIGINT NOT NULL,
  checksumSha256 VARCHAR(64) NOT NULL, containerVersion INT DEFAULT 1,
  encryptedDataKey TEXT, keyEncryptionIv VARCHAR(64),
```

uploadedAt TIMESTAMP NOT NULL

Table: FileReplicas
replicaID BIGSERIAL PK, fileID BIGINT FK -> FileMetadata.fileID,
nodeID BIGINT FK -> StorageNodes.nodeID,
status VARCHAR(20) DEFAULT 'PENDING', remotePath VARCHAR(255),
updatedAt TIMESTAMP NOT NULL

===== 12. API ENDPOINTS =====

- POST /auth/login Body: {email, password} -> {token}
- POST /auth/register Body: {name,surname,email,password} -> {name,email,token}
- GET /test -> string
- POST /files/upload MultipartFile + JWT -> {fileID, originalName, contentType, size}
- GET /files JWT -> [{file metadata + replicas}]
- GET /files/{fileId}/download JWT -> bytes (MVP: empty)
- POST /nodes/register name, totalSpaceBytes + JWT -> StorageNode
- GET /nodes JWT -> [StorageNode]
- POST /nodes/{nodeId}/heartbeat JWT -> 200/404

===== 13. BUILD & RUN =====

- ./mvnw spring-boot:run (or: ./mvnw clean package && java -jar target/*.jar)
- Server port: 5000
- DB: PostgreSQL localhost:5432/storagearchivedb (user: postgres / pass: System.out.print)
- Frontend CORS: http://127.0.0.1:5173 (Vite dev server)

===== 14. CLASS RELATIONSHIP MAP =====

Entity relationships:

- User 1->N StorageNode (via owner)
- User 1->N FileMetadata (via owner)
- Role 1->N User (via role)
- FileMetadata 1->N FileReplica (cascade ALL, orphan removal)
- StorageNode 1->N FileReplica (via node)

Dependency relationships:

- AuthController -> UserRepository, RoleRepository, PasswordEncoder, AuthenticationManager, TokenConfig
- FileController -> ContainerService, FileMetadataRepository, ReplicationService
- NodeController -> StorageNodeRepository
- ContainerService -> FileMetadataRepository
- ReplicationService -> FileMetadataRepository, FileReplicaRepository, StorageNodeRepository
- SecurityFilter -> TokenConfig
- AuthConfig -> UserRepository
- TokenConfig -> (none, uses hardcoded secret)

===== 15. BUGS, ISSUES & IMPROVEMENTS =====

CRITICAL:

- JWT secret hardcoded in source - must use env variable or key vault
- StorageNode.createdAt never set in NodeController -> NULL in DB
- Node heartbeat has no ownership verification - any user can heartbeat any node
- Replication only creates DB records - no actual file data transfer to nodes
- File download returns empty bytes - not functional

MAJOR:

- UserServiceImpl.getCurrentUser() returns null - SecurityContextHolder not wired
- isEmailUnique() naming is confusing (returns true when email EXISTS)
- ContainerService.buildContainer/extractContainer are passthrough stubs
- No WebSocket message handlers despite WebSocketConfig
- Role ID 3 hardcoded in registration - fragile, should use role name
- No validation on file size in upload endpoint
- No pagination on file list or node list endpoints

MINOR:

- System.out.println used for logging instead of proper logger
- FileMetadata.entity name 'FileMetadata' vs table 'FileMetadata' - OK but confusing
- ReplicationService uses raw class name inline instead of import

===== 16. HOW ALL CLASSES WORK TOGETHER - RUNTIME FLOW =====

STARTUP:

1. CloudsystemApplication.main() -> SpringApplication.run()
2. Spring context loads all @Configuration, @Service, @Repository, @Component beans
3. SecurityConfig creates SecurityFilterChain with SecurityFilter
4. TokenConfig bean ready with hardcoded secret
5. WebSocketConfig registers /ws-nodes STOMP endpoint

REQUEST: POST /auth/register
1. CorsConfig allows request from frontend origin
2. SecurityFilter: no Bearer token -> passes through (this endpoint is permitAll)
3. AuthController.register():
   -> RoleRepository.getReferenceById(3) loads Role
   -> PasswordEncoder.encode(password) hashes password
   -> User entity built, registrationData=now
   -> isEmailUnique() checks UserRepository.findUserByEmail()
   -> UserRepository.save() persists user (JPA -> INSERT INTO Users)
   -> TokenConfig.generateToken(user) creates JWT
   -> Returns RegisterUserResponse with JWT

REQUEST: POST /files/upload
1. SecurityFilter: reads Bearer token -> TokenConfig.validateToken()
   -> JWTUserData extracted -> SecurityContext set
2. FileController.uploadFile(@AuthenticationPrincipal User user, MultipartFile file):
   -> ContainerService.uploadFile(user, file):
      -> SHA-256 of file bytes
      -> FileMetadata entity built
      -> FileMetadataRepository.save() -> INSERT INTO FileMetadata
      -> returns fileID
   -> ReplicationService.replicateFile(fileID) [async]:
      -> FileMetadataRepository.findById(fileID)
      -> StorageNodeRepository.findAllByStatus(ONLINE)
      -> selectNodesWithSpace() picks top 3 by free space
      -> For each node: FileReplica entity with PENDING status
      -> FileReplicaRepository.save() -> INSERT INTO FileReplicas
   -> UploadResponse returned

REQUEST: GET /files
1. SecurityFilter validates JWT
2. FileController.listUserFiles(@AuthenticationPrincipal User user):
   -> FileMetadataRepository.findAllByOwner(user)
   -> Maps each FileMetadata to FileMetadataResponse
   -> For each file, iterates replicas -> ReplicaInfo(nodeID, nodeName, status)
   -> Returns JSON array

REQUEST: POST /nodes/register
1. SecurityFilter validates JWT
2. NodeController.registerNode():
   -> StorageNodeRepository.findAllByOwner(currentUser)
   -> If exists: return existing node
   -> Else: StorageNode.builder().owner(currentUser).name(name).status(OFFLINE).build()
   -> StorageNodeRepository.save() -> INSERT INTO StorageNodes
   -> Returns node (createdAt is NULL!)

REQUEST: POST /nodes/{nodeId}/heartbeat
1. SecurityFilter validates JWT
2. NodeController.heartbeat(@PathVariable Long nodeId):
   -> StorageNodeRepository.findById(nodeId)
   -> Sets status=ONLINE, lastHeartbeat=now
   -> StorageNodeRepository.save() -> UPDATE StorageNodes